

Correctness-by-Construction for Personal Task Management

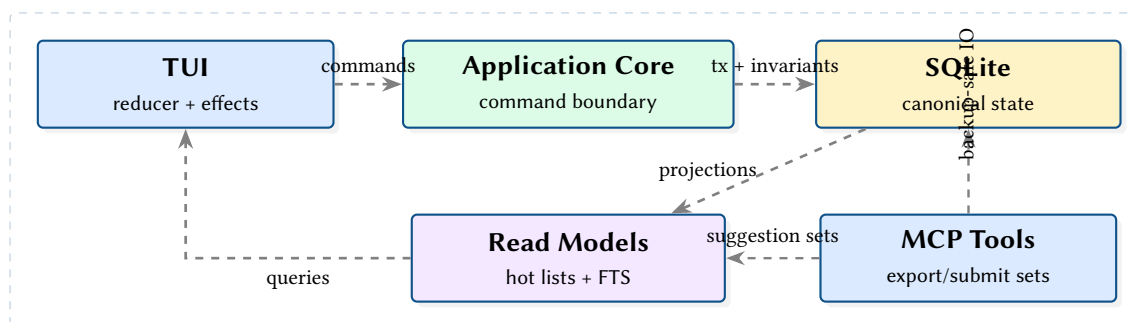
A Local-First Rust TUI GTD System with SQLite, Undo Semantics, and LLM-Ready Suggestion Sets

Simon Knight

macOS (local-first)

March 2026 • Version 0.1.0

Last updated: March 12, 2026



Dual-lane architecture: mutate via commands, observe via projections.

Abstract. This report specifies a local-first, keyboard-driven GTD task manager implemented as a Rust workspace. The system is designed around a single-writer mutation boundary and a CQRS-lite read layer, enabling sub-second interaction at ~10k actions while maintaining auditable semantics for destructive edits via undo. A one-time OmniFocus package importer provides migration into an owned SQLite schema. Finally, an MCP server exposes structured export/submit tools to support opt-in AI-assisted inbox triage through persisted suggestion sets with confidence scores and thematic grouping. Rather than presenting the system as a set of boxes, we formalize the computational ideas: availability semantics as a monotone predicate over state, tag hierarchy filtering as a transitive-closure problem, and suggestion application as a constrained batch-optimization under user-confirmation.

Contents

List of Design Decisions & Rules	2
1 Problem Statement: Speed Without Semantic Drift	3
2 Conceptual Architecture: Dual-Lane State Machine	3
3 Availability Semantics as a Monotone Predicate	3
4 Hierarchical Tags as a Transitive-Closure Problem	3
5 Importer as ETL with an Intermediate Representation	4
6 Undo as an Event-Sourced Slice	4
7 Suggestion Sets: Batch Optimization Under User Confirmation	4
8 Implementation Mapping to Workspace	4
9 Safety Model	5
9.1 Command Boundary Invariants	5
9.2 Undo Guarantees as a Restricted Rewrite System	5
List of Figures	7
List of Tables	7
Revision History	7

List of Design Decisions & Rules

1	Design Rule: Single Mutation Boundary	3
2	Design Rule: Descendant-Inclusive Semantics	4
3	Design Rule: Proposals Do Not Mutate	5

1 Problem Statement: Speed Without Semantic Drift

The core objective is not “a todo app”; it is a high-throughput transformation engine that turns unstructured captures into an actionable plan. The operational constraint is that the transformation must be fast enough to occur while attention is still available.

The system targets:

- sub-second updates for hot lists (Inbox, Next, Search) at ~10k actions;
- deterministic semantics for availability (“Next” is computed, not manually curated);
- owned schema in SQLite with migrations;
- one-time OmniFocus migration;
- opt-in, advisory-only AI suggestions expressed as persisted batches (never implicit edits).

Insight:
The bottleneck is not storage; it is the cognitive context-switch during triage. The UI must reduce the number of attention transitions per item.

: Single Mutation Boundary

All persisted changes occur through a single mutation boundary that can enforce invariants, attach provenance metadata, and generate undo records.

2 Conceptual Architecture: Dual-Lane State Machine

The architecture is most usefully understood as two coupled dynamical systems: (i) a mutation lane (commands) that changes canonical state, and (ii) an observation lane (queries) that emits projections shaped for interaction.

Mutation Lane (Commands)

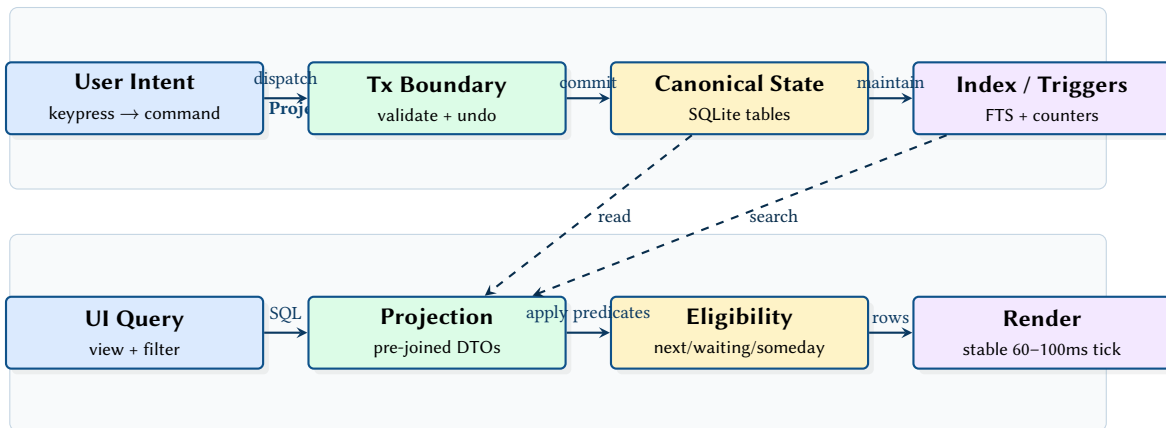


Figure 1. A dual-lane architecture: commands mutate canonical state; queries read projections optimized for interaction latency. This is closer to a control system than an object model.

3 Availability Semantics as a Monotone Predicate

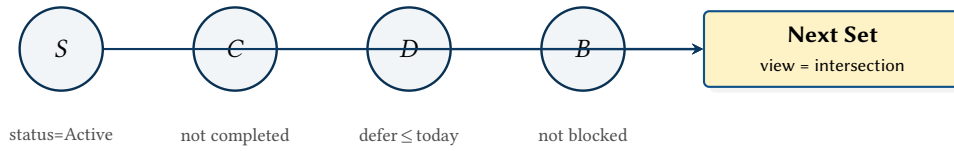
The “Next” list is not curated; it is computed. At the domain level, an action (task) is eligible iff it satisfies a predicate over state:

$$\text{eligible}(a, t) = \left(\text{status}(a) = \text{Active} \right) \wedge \left(\neg \text{completed}(a) \right) \wedge \left(\text{defer}(a) \leq t \right) \wedge \left(\neg \text{blocked}(a) \right)$$

4 Hierarchical Tags as a Transitive-Closure Problem

Tags form a rooted forest. Filtering by a tag typically includes descendants, which is a transitive-closure query. A correct implementation can use a recursive CTE; a fast implementation can optionally maintain a closure table.

Insight:
Computing eligibility as a predicate collapses multiple UI lists into a single semantics. Views become lenses, not bespoke logic.



Monotone edit intuition. Small edits move an item across a small number of predicate boundaries (e.g., setting a defer date toggles only D). This makes batch triage predictable and undoable.

Figure 2. Eligibility for the “Next” view as an intersection of predicates. This diagram emphasizes the computational structure (set intersection) rather than a linear flow.

: Descendant-Inclusive Semantics

Tag-based filtering includes all descendants of a chosen tag by default; the UI may expose a toggle, but the default matches the cognitive model (“work” includes “work/admin”).

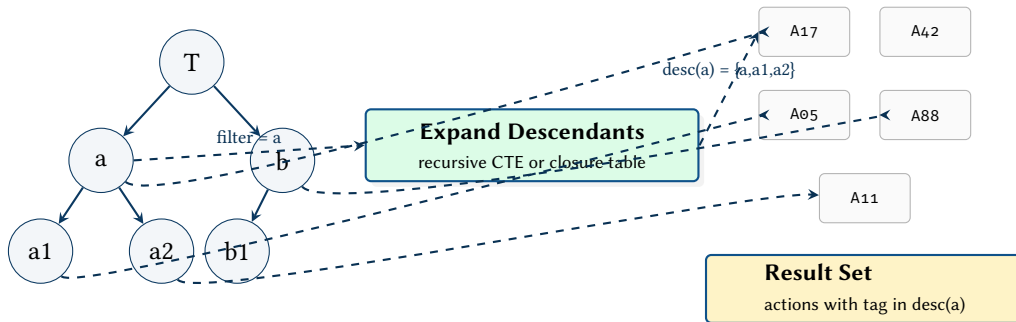


Figure 3. Tag filtering is a graph problem: compute a descendant set in the tag forest, then select actions with membership in that set. The diagram emphasizes closure expansion and set membership rather than a flowchart.

5 Importer as ETL with an Intermediate Representation

The OmniFocus importer is a one-time migration, which enables a stronger posture: the pipeline can be built for auditability and resilience rather than incremental sync.

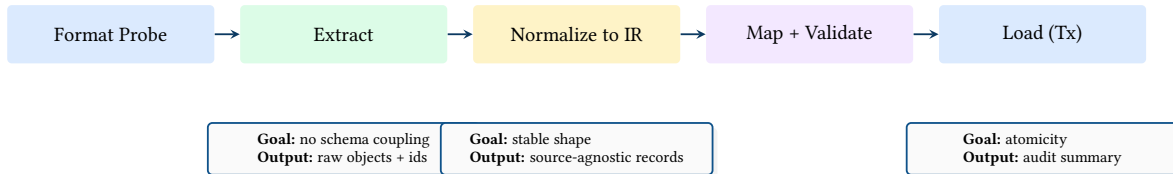


Figure 4. Import is treated as ETL with an explicit IR. The IR is the semantic compression boundary: it isolates source quirks from the domain model.

6 Undo as an Event-Sourced Slice

Undo is not implemented as “reverse SQL”; it is implemented as a constrained event log over destructive operations. This yields two desirable properties: (i) a small set of operations is reliably reversible, and (ii) the UI can treat undo as a first-class capability with predictable scope.

7 Suggestion Sets: Batch Optimization Under User Confirmation

The system intentionally does not auto-edit tasks based on AI output. Instead it persists suggestion sets: batched proposed edits with confidence and thematic grouping.

We can model suggestion acceptance as a constrained selection problem. Let i index inbox items and let k index proposed labels (project, tags, defer/due). Each proposed edit e_{ik} has confidence p_{ik} and a cost c_{ik} (edit friction). The user chooses a subset that maximizes expected utility subject to safety constraints (e.g., never change completed items; never overwrite explicit manual tags).

8 Implementation Mapping to Workspace

This report is grounded in the codebase structure:

- `crates/gsd_tui`: ratatui UI, event loop, views.

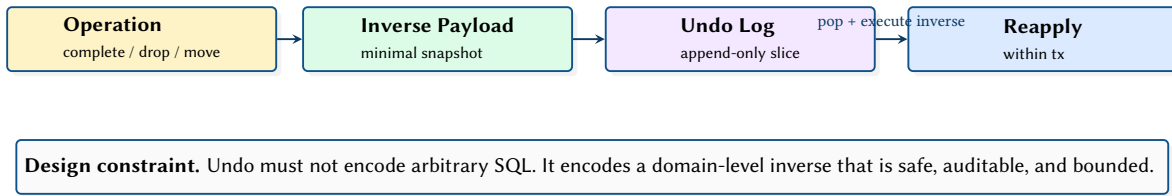


Figure 5. Undo is modeled as a bounded event-sourced slice: operations store a minimal inverse payload; undo replays the inverse inside a transaction. The intent is semantic reversibility, not perfect time travel.

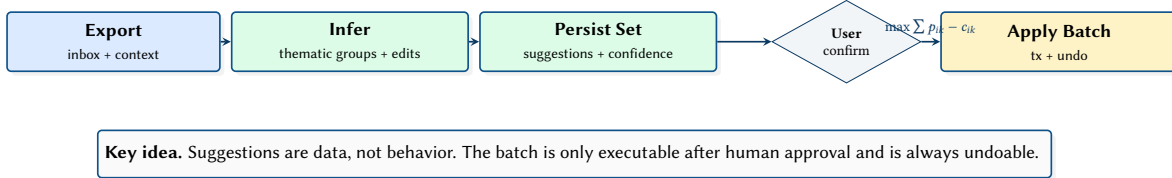


Figure 6. Suggestion sets convert probabilistic inference into a deterministic workflow: export context, infer proposals, persist a set, then apply only after user confirmation (with undo). The diagram encodes the optimization framing rather than a simple pipeline.

- `crates/gsd_domain`: core model types (actions/projects/tags) and metadata.
- `crates/gsd_infra_sqlite`: DB opening PRAGMAs, migrations, repos, search.
- `crates/gsd_undo`: undo payloads + application.
- `crates/gsd_mcp`: MCP server exposing export/submit tools for Claude Code integration.

9 Safety Model

This system is primarily optimized for *semantic integrity*: a user should be able to trust that the system's derived lists (Next/Waiting/Someday) reflect explicit state, and that any destructive operation is reversible within a bounded scope.

Insight:
The most important boundary is social, not technical: the code enforces that probabilistic systems can propose, but only deterministic commands can mutate.

9.1 Command Boundary Invariants

The command boundary is where invariants are enforced. Conceptually, state transitions are of the form $S_{t+1} = \delta(S_t, c)$ where c is a user command. The correctness strategy is to ensure δ is total over admissible commands and preserves invariants.

: Proposals Do Not Mutate

External or probabilistic systems (e.g., LLM tooling) may only create *proposal data* (suggestion sets). The only mechanism that mutates canonical state is a domain command executed within a transaction.



Safety direction. The system is a one-way membrane: proposals can flow inward as data, but cannot cross into mutation without confirmation and a deterministic command. This prevents "silent edits" and makes provenance inspectable.

Figure 7. A safety membrane: suggestion sets are persisted as data, then converted into commands only through explicit confirmation and a transaction boundary with invariants and undo.

9.2 Undo Guarantees as a Restricted Rewrite System

Undo is safest when it is not "universal". Instead, it is defined over a restricted alphabet of operations (e.g., complete, drop, move) where each operation has a defined inverse with bounded, auditable payload.

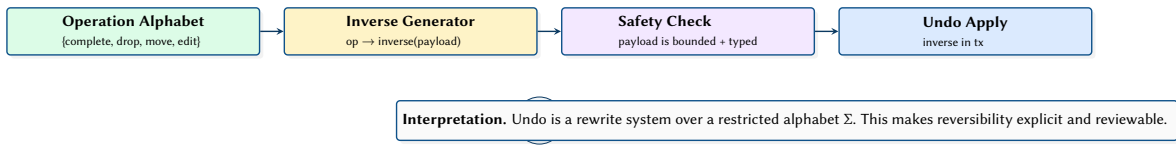


Figure 8. Undo framed as a restricted rewrite system: only operations with defined inverses participate, and inverse payloads are bounded and typed. This is a correctness move, not a performance trick.

List of Figures

1	A dual-lane architecture: commands mutate canonical state; queries read projections optimized for interaction latency. This is closer to a control system than an object model.	3
2	Eligibility for the “Next” view as an intersection of predicates. This diagram emphasizes the computational structure (set intersection) rather than a linear flow.	4
3	Tag filtering is a graph problem: compute a descendant set in the tag forest, then select actions with membership in that set. The diagram emphasizes closure expansion and set membership rather than a flowchart.	4
4	Import is treated as ETL with an explicit IR. The IR is the semantic compression boundary: it isolates source quirks from the domain model.	4
5	Undo is modeled as a bounded event-sourced slice: operations store a minimal inverse payload; undo replays the inverse inside a transaction. The intent is semantic reversibility, not perfect time travel.	5
6	Suggestion sets convert probabilistic inference into a deterministic workflow: export context, infer proposals, persist a set, then apply only after user confirmation (with undo). The diagram encodes the optimization framing rather than a simple pipeline.	5
7	A safety membrane: suggestion sets are persisted as data, then converted into commands only through explicit confirmation and a transaction boundary with invariants and undo.	5
8	Undo framed as a restricted rewrite system: only operations with defined inverses participate, and inverse payloads are bounded and typed. This is a correctness move, not a performance trick.	6

List of Tables

Revision History

Version	Date	Changes
0.1.0	March 2026	Initial technical report drafted from workspace code + planning specs.
